

GenerationMixin.generate

https://huggingface.co/docs/transformers/main/en/main_classes/text_generation

Detailed Documentation

class `transformers.GenerationConfig`

`transformers.GenerationConfig`

class

`transformers.GenerationConfig` https://github.com/huggingface/transformers/blob/main/src/transformers/generation/configuration_utils.py#L100
Parameters that control the length of the output

- `max_length` (`int`, *optional*, defaults to 20) --

The maximum length the generated tokens can have. Corresponds to the length of the input prompt +

`max_new_tokens`. Its effect is overridden by `max_new_tokens`, if also set.

- `max_new_tokens` (`int`, *optional*) --

The maximum numbers of tokens to generate, ignoring the number of tokens in the prompt.

- `min_length` (`int`, *optional*, defaults to 0) --

The minimum length of the sequence to be generated. Corresponds to the length of the input prompt +

`min_new_tokens`. Its effect is overridden by `min_new_tokens`, if also set.

- `**min_new_tokens**` (`int`, *optional*) --

`from_pretrainedtransformers.GenerationConfig.from_pretrainedh`

```
from_pretrainedtransformers.GenerationConfig.from_pretrainedhttps://github.com
"pretrained_model_name", "val": ": typing.Union[str, os.PathLike]"}, {"name":
"config_file_name", "val": ": typing.Union[str, os.PathLike, NoneType] = None"},
{"name": "cache_dir", "val": ": typing.Union[str, os.PathLike, NoneType] =
None"}, {"name": "force_download", "val": ": bool = False"}, {"name":
"local_files_only", "val": ": bool = False"}, {"name": "token", "val": ":
typing.Union[bool, str, NoneType] = None"}, {"name": "revision", "val": ": str =
'main'"}, {"name": "***kwargs", "val": ""}]- **pretrained_model_name** ( str or
os.PathLike ) --
```

This can be either:

- a string, the *model id* of a pretrained model configuration hosted inside a model repo on
- a path to a *directory* containing a configuration file saved using the

```
[ save_pretrained ()]
```

(/docs/transformers/main/en/main_classes/text_generation# `transformers.Gener`
method, e.g., `./my_model_directory/` .

- `**config_file_name**` (`str` or `os.PathLike`, *optional*, defaults to `"generation_config.json"`) --

Name of the generation configuration JSON file to be loaded from `pretrained_model_name`.

- `**cache_dir**` (`str` or `os.PathLike`, *optional*) --

Path to a directory in which a downloaded pretrained model configuration should be cached if the

standard cache should not be used.

from_model_configtransformers.GenerationConfig.from_model_c

from_model_configtransformers.GenerationConfig.from_model_confighttps://github.com/huggingface/transformers/blob/main/src/transformers/configuration_utils.py#L1000-L1010

```
"model_config", "val": "PreTrainedConfig"]- **model_config**
( PreTrainedConfig ) --
```

The model config that will be used to instantiate the generation config.
0[GenerationConfig]

(/docs/transformers/main/en/main_classes/text_generation#transformers.GenerationConfig) --
configuration object instantiated from those parameters.

Instantiates a [GenerationConfig]

(/docs/transformers/main/en/main_classes/text_generation#transformers.GenerationConfig) --

from a [PreTrainedConfig]

(/docs/transformers/main/en/main_classes/configuration#transformers.PreTrainedConfig) --

This function is useful to convert legacy

[PreTrainedConfig]

(/docs/transformers/main/en/main_classes/configuration# transformers.PreTrai

objects, which may contain generation parameters, into a stand-alone
[GenerationConfig]

(/docs/transformers/main/en/main_classes/text_generation# transformers.Gener

save_pretrainedtransformers.GenerationConfig.save_pretrainedh

save_pretrainedtransformers.GenerationConfig.save_pretrainedhttps://github.com

```
"save_directory", "val": ": typing.Union[str, os.PathLike]"}, {"name":  
"config_file_name", "val": ": typing.Union[str, os.PathLike, NoneType] = None"},  
{"name": "push_to_hub", "val": ": bool = False"}, {"name": "**kwargs", "val": ""}]-  
**save_directory** ( str or os.PathLike ) --
```

Directory where the configuration JSON file will be saved (will be created if it
does not exist).

- **config_file_name** (str or os.PathLike , *optional*, defaults to
"generation_config.json") --

Name of the generation configuration JSON file to be saved in
save_directory .

- **push_to_hub** (bool , *optional*, defaults to False) --

Whether or not to push your model to the Hugging Face model hub after saving
it. You can specify the

repository you want to push to with repo_id (will default to the name of

`save_directory` in your

- `**kwargs` (`dict[str, Any]`, *optional*) --

Additional key word arguments passed along to the [`push_to_hub()`]
(/docs/transformers/main/en/main_classes/model# `transformers.utils.PushToHub`
method.0

Save a generation configuration object to the directory `save_directory`, so
that it can be re-loaded using the

`updatetransformers.GenerationConfig.updatehttps://github.com/`

`updatetransformers.GenerationConfig.updatehttps://github.com/huggingface/tra`
`***kwargs", "val": ""}] - **kwargs` (`dict[str, Any]`) --

Dictionary of attributes to tentatively update this class.0 `dict[str, Any]`
`Any]` Dictionary containing all the key-value pairs that were not used to update
the instance.

Updates attributes of this class instance with attributes from `kwargs` if they
match existing attributes,

returning all the unused kwargs.

`validatetransformers.GenerationConfig.validatehttps://github.com`

`validatetransformers.GenerationConfig.validatehttps://github.com/huggingface/tra`

"strict", "val": " = False"]}- **strict** (bool) -- If True, raise an exception for any issues found. If False, only log issues.0

Validates the values of the attributes of the [GenerationConfig] (/docs/transformers/main/en/main_classes/text_generation# `transformers.Gener` instance. Raises exceptions in the presence

of parameterization that can be detected as incorrect from the configuration instance alone.

Note that some parameters not validated here are best validated at generate runtime, as they may depend on

other inputs and/or the model, such as parameters related to the generation length.

get_generation_modetransformers.GenerationConfig.get_generat

get_generation_modetransformers.GenerationConfig.get_generation_modehttps://
"assistant_model", "val": ": typing.Optional[ForwardRef('PreTrainedModel')] =
None"]}- **assistant_model** (`PreTrainedModel`, *optional*) --

The assistant model to be used for assisted generation. If set, the generation mode will be

assisted generation.0 `GenerationMode` The generation mode triggered by the instance.

Returns the generation mode triggered by the [GenerationConfig]

(/docs/transformers/main/en/main_classes/text_generation# `transformers.Generation` instance.

`class` `<span class="code-`

`inline">transformers.GenerationMixin``transformers.GenerationMix`

`class`

`transformers.GenerationMixin``transformers.GenerationMixin``https`://github.

A class containing all functions for auto-regressive text generation, to be used as a mixin in model classes.

Inheriting from this class causes the model to have special generation-related behavior, such as loading a

`GenerationConfig` at initialization time or ensuring `generate`-related tests are run in `transformers` CI.

A model class should inherit from `GenerationMixin` to enable calling methods like `generate`, or when it

has defined a custom `generate` method that relies on `GenerationMixin`, directly or indirectly, which

approximately shares the same interface to public methods like `generate`.

Three examples:

- `LlamaForCausalLM` should inherit from `GenerationMixin` to enable calling `generate` and other public

methods in the mixin;

- `BlipForQuestionAnswering` has a custom `generate` method that approximately shares the same interface as

`transformers.GenerationMixin.generate` <https://github.com>

```
transformers.GenerationMixin.generate
https://github.com/huggingface/tr
"inputs", "val": ": typing.Optional[torch.Tensor] = None", {"name":
"generation_config", "val": ":
typing.Optional[ transformers.generation.configuration_utils.GenerationCo
= None", {"name": "logits_processor", "val": ":
typing.Optional[ transformers.generation.logits_process.LogitsProcessorLi
= None", {"name": "stopping_criteria", "val": ":
typing.Optional[ transformers.generation.stopping_criteria.StoppingCriter
= None", {"name": "prefix_allowed_tokens_fn", "val": ":
typing.Optional[collections.abc.Callable[[int, torch.Tensor], list[int]]] = None",
{"name": "synced_gpus", "val": ": typing.Optional[bool] = None", {"name":
"assistant_model", "val": ": typing.Optional[ForwardRef('PreTrainedModel')] =
None", {"name": "streamer", "val": ":
typing.Optional[ForwardRef('BaseStreamer')] = None", {"name":
"negative_prompt_ids", "val": ": typing.Optional[torch.Tensor] = None", {"name":
"negative_prompt_attention_mask", "val": ": typing.Optional[torch.Tensor] =
None", {"name": "use_model_defaults", "val": ": typing.Optional[bool] = None",
{"name": "custom_generate", "val": ": typing.Union[str, collections.abc.Callable,
NoneType] = None", {"name": "**kwargs", "val": ""}]- **inputs** ( torch.Tensor
of varying shape depending on the modality, *optional*) --
```

The sequence used as a prompt for the generation or as model inputs to the encoder. If `None` the

method initializes it with `bos_token_id` and a batch size of 1. For decoder-only models `inputs`

should be in the format of `input_ids`. For encoder-decoder models `*inputs*` can represent any of

`input_ids`, `input_values`, `input_features`, or `pixel_values`.

- `**generation_config**` ([GenerationConfig] (/docs/transformers/main/en/main_classes/text_generation# `transformers.G` *optional*) --

The generation configuration to be used as base parametrization for the generation call. `**kwargs`

passed to generate matching the attributes of `generation_config` will override them. If

`generation_config` is not provided, the default will be used, which has the following loading

priority: 1) from the `generation_config.json` model file, if it exists; 2) from the model

`compute_transition_score` `transformers.GenerationMixin.compute`

```
compute_transition_score transformers.GenerationMixin.compute_transition_score
"sequences", "val": ": Tensor"}, {"name": "scores", "val": ": tuple"}, {"name":
"beam_indices", "val": ": typing.Optional[torch.Tensor] = None"}, {"name":
"normalize_logits", "val": ": bool = False}]- **sequences** ( torch.LongTensor )
--
```

The generated sequences. The second `dimension` (sequence_length) is either equal to `max_length` or

shorter if all batches finished early due to the `eos_token_id`.

- `**scores**` (`tuple (torch.FloatTensor)`) --

Transition scores for each vocabulary token at each generation step. Beam transition scores consisting

of l